# Experiment 6

**Student Name: Ashi Gupta** **UID: 25MCA20160**

**Branch: MCA General** **Section/Group: 25MCA-1A**

**Semester: II** **Date of Performance: 02/03/26**

**Subject Name: Technical Training** **Subject Code: 25CAP-652**

**Experiment Aim:** Learn how to create, query, and manage views in SQL to simplify database queries and provide a layer of abstraction for end-users.

**Tools used:** Oracle / MS SSMS / PostgreSQL

## Objectives:

- **Data Abstraction:** To understand how to hide complex table joins and calculations behind a simple virtual table interface.
- **Enhanced Security:** To learn how to restrict user access to sensitive columns by providing views instead of direct table access.
- **Query Simplification:** To master the creation of views that pre-join multiple tables, making reporting easier for non-technical users.
- **View Management:** To understand the syntax for creating, altering, and dropping views, as well as the naming conventions required for efficient data access.

## Experiment Steps:

```
CREATE TABLE Departments (

    dept_id INTEGER PRIMARY KEY,

    dept_name VARCHAR(50)

);

INSERT INTO Departments (dept_id, dept_name) VALUES

(1, 'HR'),

(2, 'IT'),

(3, 'Finance');

CREATE TABLE Employees (

    emp_id INTEGER PRIMARY KEY,

    emp_name VARCHAR(50),
```

city VARCHAR(50),

salary INTEGER,

status VARCHAR(20),

dept_id INTEGER REFERENCES Departments(dept_id)

);

INSERT INTO Employees (emp_id, emp_name, city, salary, status, dept_id) VALUES

(101, 'Rahul', 'Delhi', 50000, 'Active', 1),

(102, 'Anita', 'Mumbai', 60000, 'Active', 2),

(103, 'Aman', 'Chandigarh', 45000, 'Inactive', 2),

(104, 'Priya', 'Delhi', 70000, 'Active', 3);

| | emp_id [PK] integer | emp_name character varying (50) | city character varying (50) | salary integer | status character varying (20) | dept_id integer |
|---|---|---|---|---|---|---|
| 1 | 101 | Rahul | Delhi | 50000 | Active | 1 |
| 2 | 102 | Anita | Mumbai | 60000 | Active | 2 |
| 3 | 103 | Aman | Chandigarh | 45000 | Inactive | 2 |
| 4 | 104 | Priya | Delhi | 70000 | Active | 3 |

| | dept_id [PK] integer | dept_name character varying (50) |
|---|---|---|
| 1 | 1 | HR |
| 2 | 2 | IT |
| 3 | 3 | Finance |

**Step 1:** Creating a Simple View for Data Filtering

Implementing a view to provide a quick list of active employees without exposing the entire table structure.

```
CREATE VIEW Active_Employees_View AS

SELECT emp_id, emp_name, city, salary

FROM Employees WHERE status = 'Active';
```

| | emp_id integer | emp_name character varying (50) | city character varying (50) | salary integer |
|---|---|---|---|---|
| 1 | 101 | Rahul | Delhi | 50000 |
| 2 | 102 | Anita | Mumbai | 60000 |
| 3 | 104 | Priya | Delhi | 70000 |

**Step 2:** Creating a View for Joining Multiple Tables

Simplifying the retrieval of data distributed across Employees and Departments tables.

```
CREATE OR REPLACE VIEW Employee_Department_View AS
SELECT e.emp_id, e.emp_name,  e.city,  e.salary,  d.dept_name
FROM Employees e JOIN Departments d
ON e.dept_id = d.dept_id;
```

| | emp_id integer | emp_name character varying (50) | city character varying (50) | salary integer | dept_name character varying (50) |
|---|---|---|---|---|---|
| 1 | 101 | Rahul | Delhi | 50000 | HR |
| 2 | 102 | Anita | Mumbai | 60000 | IT |
| 3 | 103 | Aman | Chandigarh | 45000 | IT |
| 4 | 104 | Priya | Delhi | 70000 | Finance |

**Step 3:** Advanced Summarization View

Creating a view to provide department-level statistics automatically.

```
CREATE VIEW Department_Summary_View AS
SELECT d.dept_name,
       COUNT(e.emp_id) AS total_employees,
       AVG(e.salary) AS average_salary,
       SUM(e.salary) AS total_salary
FROM Employees e JOIN Departments d
ON e.dept_id = d.dept_id GROUP BY d.dept_name;
```

| | dept_name character varying (50) | total_employees bigint | average_salary numeric | total_salary bigint |
|---|---|---|---|---|
| 1 | Finance | 1 | 70000.000000000000 | 70000 |
| 2 | IT | 2 | 52500.000000000000 | 105000 |
| 3 | HR | 1 | 50000.000000000000 | 50000 |

## Outcomes:

- **Abstraction Proficiency:** Students will be able to create and query views to simplify efficient data access and abstraction.
- **Security Implementation:** Students will understand how to use views for data masking and providing restricted access to sensitive information.
- **Syntactic Accuracy:** Students will demonstrate the correct syntax for creating and querying views, ensuring logical clarity in naming conventions.
- **Real-world Application:** Students will be able to design views for practical domains like Library Management Systems or Payroll Systems to demonstrate functionality.